

JavaScript Arrays

GRG, HOA, WZR

Web- and Mobile Computing

5. März 2026

Arrays

Arrays sind

- geordnete Liste von Werten
- dynamisch vergrößerbar
- können unterschiedliche Datentypen enthalten
- Objekte!

Beispiel:

```
1      array = ["a", 7, true]; // [ 'a', 7, true ]
```

Die Länge eines Arrays `a` erhält man über die Eigenschaft `a.length`.

Erstellen von Arrays

Array Literal

```
const numbers = [1, 2, 3];
```

Konstruktor

```
const numbers = new Array(1, 2, 3);
```

Leeres Array

```
const empty = [];
```

Ein leeres Array mit n Plätzen:

```
const a = new Array(n);
```

Iterieren über Arrays

Iteratoren:

- `a.values()`: Werte des Arrays
- `a.keys()`: Schlüssel des Arrays
- `a.entries()`: (Index, Wert)-Paare

Iteration über die Werte:

```
const a = ['a', 'b', 'c', 'd'];  
for (const x of a) {  
    console.log(x);  
}  
// Ausgabe: a, b, c, d
```

Iteration über die Schlüssel:

```
const a = ['a', 'b', 'c', 'd'];  
for (const x in a) {  
    console.log(x);  
}  
// Ausgabe: 0, 1, 2, 3
```

Funktionen

Beispiele mit

```
array = ["a", 7, true]
```

- `array.push(x)`: hängt `x` am Ende an
- `array.pop()`: entfernt und returned das letzte Element
- `array.unshift(x)`: fügt `x` am Anfang ein
- `array.shift()`: entfernt und returned das erste Element
- `array.at(index)`, gleich wie `array[index]`
- `array.concat(array2)`, modifiziert nicht, returned das zusammengekettete
- ```
Array.from(array.entries())
// liefert: [[0, 'a'], [1, 7], [2, true]]
```

# JavaScript Arrays: Spread-Operator ...

**Syntax:** [...array] bzw. [...array1, ...array2]

## Beispiele:

```
1 const tiere = ["Hund", "Katze", "Maus"];
2 const farben = ["Rot", "Blau"];
3
4 const kopie = [...tiere];
5 const kombiniert = [...tiere, ...farben];
6 const mitAnfang = ["Papagei", ...tiere];
7
8 console.log(kopie); // ["Hund", "Katze", "Maus"]
9 console.log(kombiniert); // ["Hund", "Katze", "Maus", "Rot", "Blau"]
10 console.log(mitAnfang); // ["Papagei", "Hund", "Katze", "Maus"]
```

## Erläuterung:

- Entpackt Elemente eines Arrays an der gewünschten Stelle
- Sehr praktisch zum Zusammenhängen und Einfügen in neue Arrays
- Erzeugt ein neues Array (Originale bleiben unverändert)

# Funktionen

- `array.filter(cb)`: Neues Array mit allen Elementen wo `cb` `true` ergibt.
- `array.find(cb)`: liefert das element oder `undefined` (`cb`: `boolean`)
- `array.findIndex(cb)`: wie oben, aber `Index`
- `array.findLast(cb)`: wie gehabt, aber von hinten
- `array.findLastIndex()`: ebenso.
- `array.flat()`: wenn das array sub-arrays hat, werden diese "aufgemacht", das returnete array hat keine sub-arrays mehr.
- `array.flatMap()`
- `array.forEach(cb)`: `cb` wird für jedes Element aufgerufen
- `array.includes(elem)`: `boolean`
- `array.indexOf(elem)`: `index`
- `array.join(sepStr)`: array wird zu einem String zusammengejoinet
- `array.keys()`: Iterator über Indexe (`for` (`let i of array.keys()`))
- `array.lastIndexOf(elem)`
- `array.map(cb)`: Neues Array mit jedem return-wert von `cb`

# JavaScript Arrays: forEach

**Syntax:** `array.forEach((element, index, array) => { ... })`

## Beispiel: Ausgabe jedes Elements

```
1 const fruits = ["Apple", "Banana", "Cherry"];
2
3 fruits.forEach((fruit, idx) => {
4 console.log(`${idx}: ${fruit}`);
5 });
6
7 /* Ausgabe:
8 0: Apple
9 1: Banana
10 2: Cherry
11 */
```

## Erläuterung:

- Iteriert über jedes Element des Arrays
- Callback erhält: `element`, optional `index` und `array`
- Gibt keinen Wert zurück → nur für Seiteneffekte geeignet
- Kann **\*\*nicht vorzeitig abbrechen\*\*** (kein `break`)



# Funktionen

- `array.pop()`: entfernt und returned das letzte Element
- `array.push(elt)`: hängt elt an
- `array.reduce()`
- `array.reverse()`: inplace!
- `array.shift()`: entfernt und returniert das elt mit index 0.
- `array.slice()`:
- `array.some(cb)`: neues array mit allen elementen wo cb true war.
- `array.sort(cb)`: inplace sort. cb returned `j0`, 0, oder `!0`.
- `array.splice()`
- `array.toLocaleString()`
- `array.toReversed()`: returned reversed
- `array.toSorted(cb)`: returned sortiert, siehe `sort()`
- `array.toSpliced()`
- `array.toString()`: String des arrays, siehe auch `join()`
- `array.unshift(elt)`: fügt elt an den index 0 ein

# Funktionen

**Syntax:** `array.splice(start, deleteCount, item1, item2, ...)`

**Beispiel:**

```
1 const arr = [1, 2, 3, 4, 5];
2
3 // Entferne 2 Elemente ab Index 1 und fuege 9 und 10 ein
4 arr.splice(1, 2, 9, 10);
5
6 console.log(arr); // [1, 9, 10, 4, 5]
```

**Erklärung:**

- `start`: Index, ab dem Elemente entfernt/ersetzt werden
- `deleteCount`: Anzahl der zu entfernenden Elemente
- Weitere Argumente: neue Elemente, die eingefügt werden

# JavaScript Arrays: slice

**Syntax:** `array.slice(start, end)`

**Beispiel:**

```
1 const arr = [1, 2, 3, 4, 5];
2
3 // Extrahiere Elemente von Index 1 bis 3 (exklusiv)
4 const subArr = arr.slice(1, 3);
5
6 console.log(subArr); // [2, 3]
7 console.log(arr); // [1, 2, 3, 4, 5] Original bleibt unverändert
```

**Erläuterung:**

- `start` → inklusiver Startindex
- `end` → exklusiver Endindex (bis, aber nicht einschließlich)
- Gibt ein neues Array zurück → Original-Array wird nicht verändert
- Kann auch nur Start angeben: `arr.slice(2)` → alles ab Index 2

# JavaScript Arrays: reduce

**Syntax:** `array.reduce((accumulator, currentValue) => ..., initialValue)`

## Beispiel: Summiere alle Zahlen

```
1 const numbers = [1, 2, 3, 4, 5];
2
3 const sum = numbers.reduce((acc, val) => acc + val, 0);
4
5 console.log(sum); // 15
```

## Erläuterung:

- accumulator → sammelt das Ergebnis
- currentValue → aktuelles Array-Element
- initialValue → Startwert für den Akkumulator
- Reduziert das Array auf **einen einzigen Wert**

# JavaScript Arrays: map

**Syntax:** `array.map((element, index, array) => { ... })`

## Beispiel: Quadriere alle Zahlen

```
1 const numbers = [1, 2, 3, 4, 5];
2
3 const squares = numbers.map(num => num * num);
4
5 console.log(squares); // [1, 4, 9, 16, 25]
6 console.log(numbers); // [1, 2, 3, 4, 5] Original bleibt unverändert
```

## Erläuterung:

- Erstellt ein **\*\*neues Array\*\*** mit den Ergebnissen der Callback-Funktion
- Callback erhält: `element`, optional `index` und `array`
- Original-Array bleibt **\*\*unverändert\*\***
- Ideal, um **\*\*Transformationen auf Arrays\*\*** anzuwenden